SLE-2: BFS vs DFS Performance Profiling

Course: 02AML204 – Introduction to Artificial Intelligence **Program:** SY B.Tech. CSE (AI & ML) **Semester:** VI

Student: Shreya **Repository:** AI_SLE

1. Problem Statement

Breadth-First Search (BFS) and Depth-First Search (DFS) were implemented to solve the same 5 × 5 maze. Their practical performance was compared using execution time, number of nodes expanded, path length, and profiling information.

2. Objective

• Implement BFS and DFS for the same maze-search problem.

• Measure practical execution time for repeated runs.

• Compare the number of nodes expanded by both algorithms.

• Compare the resulting path length.

• Use profiling tools to observe where execution time is spent.

3. Experimental Setup

Parameter	Value
Maze size	5 × 5
Start node	(0, 0)
Goal node	(4, 4)
Path representation	0 = Path, 1 = Wall
Benchmark repetitions	50,000
Profiling tools	py-spy and cProfile
Python	Python 3.14 (64-bit)

4. Maze and Algorithm Comparison

Both algorithms were executed on the same maze, with the same start and goal. BFS uses a queue (FIFO), while DFS uses a stack (LIFO). The benchmark records the average time per execution after repeated runs.

5. Recorded Benchmark Results

Metric	BFS	DFS
Average time per run (ms)	0.025739	0.013439
Nodes expanded	17	9
Path length (path nodes)	9	9

Observation: In this recorded run, DFS required less average execution time and expanded fewer nodes, while both algorithms produced the same path length. These values are specific to the tested maze and computer environment and may vary slightly between runs.

6. Profiling Analysis

The program was profiled using py-spy. The command used was `py-spy record -o profile.svg -- python benchmark.py`. The profiler successfully generated `profile.svg` with sampled stack information. cProfile was also used in `profile_search.py` to inspect function-level execution.

Profiling Flamegraph (profile.svg)

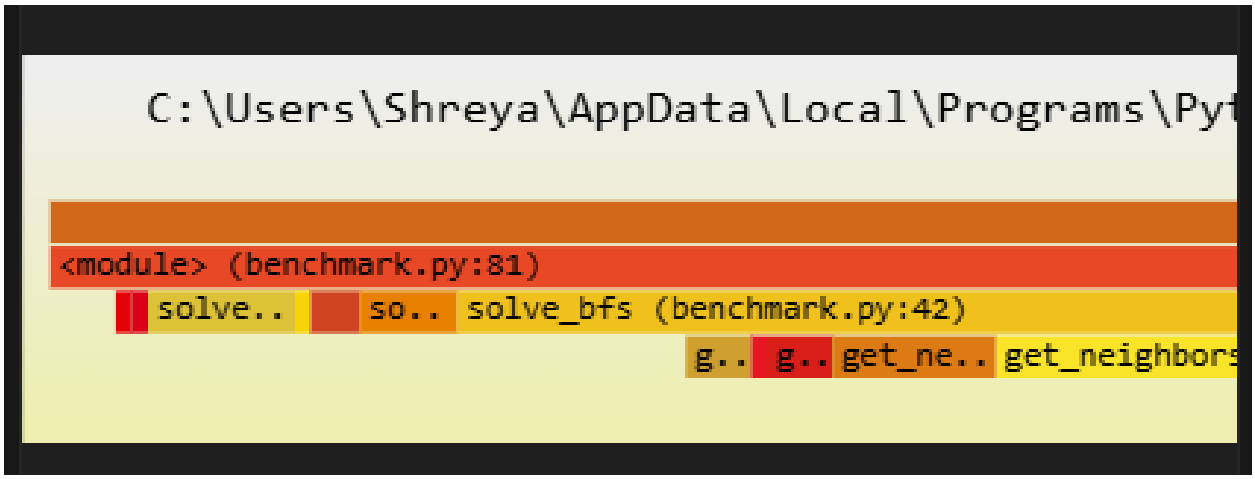

Figure 1. Profile.svg flamegraph preview generated during the profiling workflow.

7. Result and Analysis

• Both BFS and DFS successfully reached the goal node.

• The recorded path length was 9 path nodes for both algorithms.

• BFS expanded 17 nodes, whereas DFS expanded 9 nodes for this maze.

• The recorded average time was 0.025739 ms for BFS and 0.013439 ms for DFS.

• The profiling output provides empirical evidence about the functions and operations active during execution.

• The result should not be generalized to every maze: BFS and DFS can behave differently depending on maze structure, branching, and goal location.

8. Conclusion

The experiment demonstrated empirical performance analysis of two search algorithms on the same maze. For the recorded test case, DFS used less time and expanded fewer nodes, while both methods returned a path of the same length. Profiling with py-spy and cProfile helped visualize and inspect the execution behavior of the search implementation.

9. Project Files

File	Purpose
bfs_dfs.py	BFS and DFS implementation
benchmark.py	Repeated benchmark and comparison
profile_search.py	cProfile-based profiling
profile.svg	py-spy flamegraph
bfs_profile.prof / dfs_profile.prof	Profiler output files
README.md / Contribution_Log.md	Project documentation